APP DEVELOPMENT PROJECT (Report)

Trekking Management Application

MAD1 Project – BS in Data Science and Applications,
Indian Institute of Technology, Madras

Student Details :

Name: Muskan Kesherwani
Roll Number: 24f1000937
Email: 24f1000937@ds.study.iitm.ac.in

My Intro :

As an IIT student, I'm very passionate about my studies and projects, always excited to learn creative things. I am a diploma level student at IIT Madras, pursuing BS in Data Science and Applications.

Project Details :

Trekking Management Application.

Problem Statement :

A multi-role **Trekking Management Application** designed for Admins, Staff, and Users to interact with the system based on their responsibilities.

Approach :

The Trekking Management Application is developed using the Flask framework (Python) for backend development, HTML/CSS for the frontend interface, and a relational database for persistent data storage. The system follows a role-based architecture, where three types of users — Admin, Staff, and Users — are provided with distinct dashboards and permissions based on their roles.

AI/LLM Declaration :

I, Muskan Kesherwani, hereby declare that this project has been developed mainly through my own understanding, coding effort, and hands-on implementation. AI tools such as ChatGPT were utilized solely for the following purposes:

- Resolving coding errors and interpreting error messages
- Clarifying concepts related to Python, Flask, SQLAlchemy, and Jinja, and gaining a better understanding of key backend routes
- Enhancing the clarity and quality of documentation
- Assisting in refining UI/UX concepts and organizing the report content and code structure

Based on this usage, I estimate that AI/LLM assistance accounted for approximately 15–20% of the overall work, confined strictly to conceptual explanation, debugging support, and documentation improvement. I am aware of IIT Madras's academic guidelines concerning the ethical and responsible use of AI, and I affirm that this project represents my own

independent effort, with AI serving only as a supplementary tool for learning and refinement.

Technologies and Frameworks

Flask :
Serves as the primary backend framework for handling web application logic.

SQLAlchemy :
Acts as the ORM (Object Relational Mapper) for interacting with the SQLite database.

Jinja2 :
Templating engine used to generate dynamic HTML content.

Bootstrap :
Provides frontend styling, responsive layout, and overall UI design.

HTML + CSS :
Used for structuring and styling the pages visible to end users.

Font :
Supplies icon sets used across buttons, actions, and various UI elements.

Database Schema / ER Diagram (Tables)

User :

Stores trekker login credentials, profile & role information (id, name, email, password, phone, address, role, start_date, end_date, status).

Staff :

Stores staff login credentials and profile details (id, name, email, password, phone, address, status).

Trek :

Stores details of available trek (id, name, location, difficulty, duration, available_slots, start_date, end_date, status, description, phone).

Treks (Contact/Inquiry table) :

Stores messages submitted via contact form (id, name, email, subject, message, created_at).

Bookings :

Stores trek booking details linking users to treks (id, user_id, trek_id, booking_date, status)
--Foreign Keys: user_id → User(id), trek_id → Trek(id)

Relationship

One-to-Many :

- One user can have many bookings.
  User -> Bookings

- One trek can have many bookings.
  Trek -> Bookings

Many-to-One :

- Many bookings can belong to one user.
  Bookings -> User

- Many bookings can belong to one trek
  Bookings -> Trek

Many-to-Many :

- Conceptually User and Trek have a many-to-many relationship because a user can book multiple treks and a trek can have multiple users.
- This many-to-many relationship is implemented through the Bookings junction table.

ER-Diagram



Diagram credit : https://dbdiagram.io

Authentication & Authorization

Component	Purpose
Flask `session`	Stores logged-in user's role/session
`admin_required`	Restricts routes to Admin
`staff_required`	Restricts routes to Staff
`user_required`	Restricts routes to User
`/login`	Common login page
Admin login	Pre-existing admin credentials
Staff registration	Self-registration
User registration	Self-registration
Staff status	Pending → Approved → Blacklisted
User status	Active/Blacklisted

Architecture and Features

/app.py :

Main Flask application entry point.

/database.py :

Using to store tables.

/templates :

Jinja2 HTML templates for all pages.

/database :

Database configuration and initialization.

/static :

CSS file, images.

Main Roles :

Admin, Staff, Users

Admin :

Admin is the pre-existing superuser of the application :

- Can add, edit, or remove trek
- Can approve and blacklist trek staff
- Can assign staff to a particular trek

- Can view all the users, staff, and treks
- Can search treks, staff, or users by name or ID
- Can blacklist users or staff if required

Staff :

staff members that carry out trekking and guide trekkers

- Can register and log in
- Need admin’s approval to access the dashboard
- Can view assigned treks by Admin
- Can update trek details such as:
- Available slots
- Trek status (Open/Closed)
- Can view list of users registered for their treks

Users :

- A participant who books and participates in trekking activities.
- Can register and log in
- Can view approved/open treks

- Can search and filter treks based on difficulty, location
- Can book treks
- Can view booking status and trekking history

Database Tables

Table	Primary Key	Main Purpose
`user`	`id`	Stores trekker/user accounts
`staff`	`id`	Stores trek staff accounts
`trek`	`id`	Stores trekking routes/events
`bookings`	`id`	Stores user trek bookings
`contact_messages`	`id`	Stores contact/enquiry messages

Jinja2 Concept

Jinja2 Feature	Where Used	Purpose
`{% extends %}`	Templates	Inherit `base.html`
`{% block content %}`	Templates	Insert page content

`{% for %}`	Tables	Display multiple records
`{% if %}`	Status/action sections	Conditional rendering
`{% elif %}`	Status handling	Multiple conditions
`{% else %}`	Status handling	Default condition
`{{ variable }}`	Templates	Display data
`{{ "%03d"|format(id) }}`	Booking/Trek IDs	Format IDs like `BK012`
`url_for()`	Links	Generate Flask URLs
Query parameters	Booking filters	`?status=Pending`

Flask Concept

Flask Concept	Purpose
`Flask`	Web application framework
`@app.route()`	Define URLs
`render_template()`	Render Jinja HTML
`request`	Read form/query data
`redirect()`	Redirect after actions
`session`	Login/session management
`flash()`	Display success/error messages

`url_for()`	Generate URLs
`sqlite3`	Database connection
`@wraps`	Preserve decorated function metadata
Custom decorators	Role-based access control

HTML/Bootstrap Concept

Feature	Purpose
HTML forms	Login, registration, trek creation/editing
Tables	Users, staff, treks, bookings
Bootstrap cards	Dashboard statistics
Bootstrap buttons	Actions
Badges	Status display
`table-responsive`	Mobile-friendly tables
`shadow-sm`	Card styling
`row` / `col-md-*`	Responsive layout
`form-control`	Form fields
`form-select`	Dropdowns
Navbar	Application navigation

FrameWorks / Technologies

Technology	Used For
Python	Backend programming
Flask	Web framework
Jinja2	Server-side templating
HTML	Page structure
CSS	Styling
Bootstrap	Responsive UI
SQLite	Database
sqlite3	Python-SQLite connection
JavaScript	Not required for core functionality

Trek Management

Feature	Route/Page	Role
View treks	`/admin/treks`	Admin
Edit trek	`/admin/edit/trek/<id>`	Admin
Delete trek	`/admin/treks/delete/<id>`	Admin
Open trek	`/admin/open-trek/<id>`	Admin

Create trek	`/admin/treks/add`	Admin
Close trek	`/admin/close-trek/<id>`	Admin
View available treks	User trek page	User
Search trek	Trek search	User/Admin

Trek Status

Status	Meaning
`Pending`	Trek awaiting approval
`Approved`	Trek approved
`Open`	Users can book
`Closed`	No further bookings
`Completed`	Trek has finished

Booking Status

Status	Meaning
`Pending`	Booking submitted
`Approved`	Booking accepted

Rejected	Booking rejected
Cancelled	User cancelled
Completed	Trek/booking completed

Template / Features

Template	Role / Purpose
about.html	General About page
admin_bookings.html	Admin — view/manage all bookings
admin_contact_messages.html	Admin — view contact messages
admin_dashboard.html	Admin dashboard
admin_edit_trek.html	Admin — edit trek
admin_staff.html	Admin — manage staff
admin_treks_add.html	Admin — add trek
admin_treks.html	Admin — view/manage treks
admin_users.html	Admin — view/manage users
base.html	Common layout/navbar
contact.html	Contact form
home.html	Home page
login.html	Login

`register.html`	User registration
`staff_bookings.html`	Staff — view/manage bookings
`staff_dashboard.html`	Staff dashboard
`staff_password_change.html`	Staff — change password
`staff_profile_edit.html`	Staff — edit profile
`staff_profile.html`	Staff — view profile
`staff_trek.html`	Staff — trek-related page
`user_bookings.html`	User — own bookings
`user_change_password.html`	User — change password
`user_dashboard.html`	User dashboard
`user_edit_profile.html`	User — edit profile
`user_history.html`	User — trekking/booking history
`user_profile.html`	User — view profile
`user_trek_details.html`	User — trek details
`user_trek.html`	User — trek listing

Recorded Video : Click here